

Tema 10: Planificacion de Procesos y Concurrency

Este tema combina dos areas fundamentales. La planificacion decide que proceso se ejecuta y cuando. La concurrencia aborda los problemas de multiples procesos o hilos accediendo a recursos compartidos.

Planificacion de procesos

El planificador del sistema operativo decide que proceso se ejecuta en cada momento. Los criterios que usa son: maximizar el uso de CPU, minimizar el tiempo de espera, minimizar el tiempo de respuesta para procesos interactivos, y garantizar que todos los procesos avancen.

El algoritmo mas basico es FCFS, First Come First Served, o primero en llegar primero en ser atendido. Los procesos se ejecutan en el orden en que llegan. Es simple pero puede causar el efecto convoy: si un proceso largo llega primero, todos los demas esperan detras aunque sean cortos.

El algoritmo SJF, Shortest Job First, ejecuta primero el proceso mas corto. Minimiza el tiempo de espera promedio, pero tiene un problema: puede causar hambruna, donde los procesos largos nunca se ejecutan si continuamente llegan procesos cortos.

El algoritmo Round Robin da a cada proceso un cuanto de tiempo fijo. Cuando el cuanto se agota, el proceso se pone al final de la cola y el siguiente proceso se ejecuta. Esto es justo y funciona bien para procesos interactivos.

Pregunta de examen muy importante: si aumentas mucho el cuanto de Round Robin, degenera en FCFS y causa efecto convoy. Esto tiene sentido: si el cuanto es tan grande que ningun proceso lo agota, cada proceso se ejecuta completo antes de ceder la CPU, que es exactamente FCFS.

Otra pregunta: si los cambios de contexto se hacen mas lentos, el algoritmo mas afectado es Round Robin con cuanto pequeño, porque es el que mas cambios de contexto realiza. Con cuanto pequeño, cambias constantemente de proceso, y cada cambio tiene un coste. Si ese coste aumenta, el overhead se dispara.

El comando renice de Linux permite cambiar la prioridad de un proceso en ejecucion. Los valores de niceness van de menos veinte, que es la maxima prioridad, a diecinueve, que es la minima. Solo root puede asignar prioridades negativas.

Condiciones de carrera

Una condicion de carrera ocurre cuando el resultado de un programa depende del orden en que se ejecutan los procesos o hilos. Es el peor tipo de bug porque no es reproducible y ocurre aleatoriamente. Es especialmente grave con memoria compartida.

El ejemplo clasico: dos procesos incrementan una variable compartida x. Cada uno ejecuta x mas mas mil veces. El resultado esperado es dos mil, pero el resultado real puede ser cualquier valor entre dos y dos mil.

La razon es que x mas mas no es atomico. Aunque parece una sola operacion, el procesador la ejecuta en tres pasos: primero lee el valor de x en un registro, luego incrementa el registro, y finalmente escribe el registro de vuelta a x. Si dos procesos ejecutan estos pasos intercalados, pueden pisar los incrementos del otro.

Pregunta de examen tipica: te muestran un trozo de codigo con un incremento compartido y te preguntan que puede pasar. La respuesta suele ser condicion de carrera, y el valor puede ser menor que el esperado.

Seccion critica y exclusion mutua

La seccion critica es la parte del codigo donde se accede a un recurso compartido. Solo un proceso debe estar en la seccion critica a la vez. La exclusion mutua es el mecanismo que garantiza esto.

Mecanismos de sincronizacion

El mecanismo mas basico es el test and set, que es una operacion atomica de hardware. Lee un valor y lo pone a verdadero en una sola instruccion que no puede ser interrumpida. Si el valor anterior era falso, has conseguido el cerrojo. Si era verdadero, alguien mas lo tiene y debes reintentar.

Un lock o cerrojo es una abstraccion construida sobre test and set. La funcion acquire intenta conseguir el lock en un bucle usando test and set. La funcion release simplemente pone el lock a cero. El problema es la espera activa: el proceso en el bucle consume CPU sin hacer trabajo util.

Un mutex es un lock mejorado. La diferencia es que cuando un proceso no puede conseguir el mutex, se duerme en vez de hacer espera activa. Cuando el mutex se libera, se despierta uno de los procesos esperando. Esto es mucho mas eficiente para esperas largas.

Pregunta de examen: los spinlocks no se deben usar cuando la contencion es alta, porque desperdician CPU en espera activa. Son aceptables cuando la contencion es baja y la seccion critica es muy corta, porque el coste de dormir y despertar procesos seria mayor que la espera activa.

Un semaforo es un contador con operaciones atomicas wait y signal. Wait decrementa el contador, y si el resultado es negativo, el proceso se bloquea. Signal incrementa el contador, y si habia procesos bloqueados, despierta uno. Un semaforo inicializado a uno funciona como mutex. Inicializado a otros valores permite controlar el acceso concurrente a recursos con multiples instancias.

Las variables de condicion permiten que un proceso espere hasta que se cumpla una condicion. La operacion wait libera el lock asociado y duerme al proceso. La operacion signal despierta un proceso. La operacion broadcast despierta a todos. Siempre deben usarse dentro de un bucle while que comprueba la condicion, no un if, porque puede haber despertares espurios.

Un monitor es una abstraccion de mas alto nivel que encapsula datos privados, metodos

sincronizados y variables de condicion. Todos los metodos del monitor se ejecutan con exclusion mutua implicita.

Errores tipicos de sincronizacion

El error mas comun en examenes es poner la comprobacion fuera del lock. Ejemplo: compruebas si un contador es mayor que cero, luego adquieres el lock, decrementas el contador y liberas el lock. El problema es que la comprobacion ocurre fuera del lock, asi que dos hilos pueden pasar la comprobacion simultaneamente y ambos decrementar.

La correccion es simple: pon la comprobacion dentro del lock. Primero adquiere el lock, luego comprueba la condicion, actua, y finalmente libera el lock.

Otra variante que sale en el examen: tienes una funcion que comprueba si un cliente existe en una lista y luego lo borra, pero la comprobacion esta fuera del spinlock y el borrado dentro. Es una condicion de carrera porque entre la comprobacion y el borrado, otro hilo podria haber modificado la lista.

Problemas clasicos de sincronizacion

El problema del productor-consumidor: un productor genera datos y los pone en un buffer, un consumidor los toma del buffer. Si el buffer esta lleno, el productor espera. Si esta vacio, el consumidor espera. Se resuelve con un mutex y dos variables de condicion, o con dos semaforos.

El problema de lectores-escritores: multiples lectores pueden acceder a un recurso simultaneamente, pero un escritor necesita acceso exclusivo. Si hay lectores activos, los escritores esperan. Si hay un escritor activo, todos los demas esperan.

El problema de los filosofos comensales: cinco filosofos sentados en una mesa circular con cinco palillos. Cada filosofo necesita dos palillos para comer. Si todos toman el palillo de la izquierda simultaneamente, nadie puede tomar el de la derecha y se produce un interbloqueo. Las soluciones incluyen limitar a cuatro el numero de filosofos que pueden intentar comer simultaneamente, o hacer que un filosofo tome los palillos en orden distinto.

Interbloqueo o deadlock

Un interbloqueo ocurre cuando dos o mas procesos se esperan mutuamente y ninguno puede avanzar. Las cuatro condiciones necesarias son: exclusion mutua, retencion y espera, no desalojo, y espera circular. Si eliminas cualquiera de estas condiciones, no puede haber interbloqueo.

Ejemplo simple: el proceso A tiene el recurso X y espera el recurso Y. El proceso B tiene el recurso Y y espera el recurso X. Ambos estan bloqueados para siempre.